Project: Smart Parking Platform

Company: $oftware Corp.

Name: Azeneth Castaneda

Date: 09/10/2026

I. Research: Comparative Analysis

The Smart Parking Platform will allow drivers to search for and find parking and then reserve and pay for parking in real time. Some applications have already implemented a handful of these features, but none combined all the most important aspects of parking into a single integrative payment and navigation system. ParkMobile is a mobile payment system, as well to extend your parking time without running back to the meter.

Still, it is largely an entirely payment-oriented system with limited real-time garage info & reservations. Our system will offer all four of these features.

Spot Hero offers the ability to search for and book parking in advance, mainly near events, airports, and high-traffic urban locations. Although the model is first and foremost operator-focused, it lacks comprehensive real-time occupancy tools. Our approach will benefit both drivers and operators. Park Whiz allows users to compare and place reservations at parking lots by location and price.

Our challenge is a lack of comprehensive, real time space tracking, live directions assistance, and reporting analytics. Our service offers real-time availability directions, receipts, and reporting. Passport Parking is very good for the mobile payment side of parking (used by cities, universities, and parking operators) but has not been very focused on finding free garage spaces or reserving spaces. We will include search reservation maps, alerts and management systems too.

Google Maps parking functionalities are designed to provide parking information to the car driver with search of parking places and navigation. Google Maps is not designed for the reservations and payment of parking. Our system will reuse the navigation support but will add reservation payment receipts, availability dashboard and operator dashboard functionalities.

II. Vision and Scope

The purpose of the Smart Parking Platform (SPP) is to develop a web-based and mobile-based application system that enables the drivers to locate, reserve, navigate and make payments for parking efficiently. The need for the business is that the drivers spend valuable time to search parking space, and because of this the situation causes aggravation and increases congestion.

The other business need is for parking operators to have more appropriate tools to monitor the parking occupancy, control the parking entrance fees, and increase the utilization of the parking facilities. Project scope encompasses these features and functionalities: user registration and login, live parking space online, map overlay with interactive parking location reservation online payment, reservation history, parking receipts notifications external maps overlay, administration

Significant project stakeholders encompass drivers, parking and transportation facility management, city transportation authorities, system administrators and operators, billing system administrators, their billing staff, third-party gateway via partner payment agency, development teams of all applications involved including the billing system. Considerations are: semester project's fixed scope, schedule and budget.

III. **Drafts for Software Requirements Specification**:

3.1 – Introduction

This document defines the initial software requirements document for the Smart Parking Platform. The system will cater for both the needs of parking drivers and parking operators in occupancy management, reservations management, fee payments and reporting needs

3.2 - Overall Description

The Smart Parking Platform will be offered in two formats: Web-based application and mobile based application. Drivers will use the system to identify reserve go-to and pay and report parking. Parking operators will use the system for overseeing parking limited capacity and its utilization, generate reports, provide data for estimating parking capacity and other predictive data and manage parking facility details. Users: Drivers, parking operators, city transportation departments, system administrators, billing and payment system administrators, service providers and developers. System Environment: Web based application and mobile based application. Constraints: semester system time constraints, third-party mapping and payment related API constraints, constraints related to security and privacy. Assumptions: motorists have internet access, the parking data is updated regularly, the navigation services and payments are available continuously and reliability.

3.3 – Functional Requirements

FR1: The system shall authorize the parking system based on login credentials provided by users

FR2: The system shall facilitate search of parking resources near a location of interest to a driver

FR3: The system shall provide real time parking display

FR4: The system shall allow reservation of parking resources by a driver

FR5: The system shall facilitate payments via digital means

FR6: The system shall generate a payment receipt and general reservation confirmation invoice for users.

FR7: The system shall establish notification and warning messages for users

FR8: The system shall enable search facility of parking occupancy reports by parking operators 3. 4 Non-Functional Requirements

3.4 – Non-Functional Requirements

NFR1: The system should be user-friendly both in web oriented and mobile oriented environments

NFR2: The system should provide expected degree of security and data safety among account and payment holders

NFR3: The parking resources data should be timely, accurately and consistently updated

NFR4: The system should be operational and reliable in peak cluster hours

NFR5: The system should rely on third-party API of navigation and payment processing

3.5 – Use Cases

Use Case 1: Sign in and Sign Up system

Actor: Driver

Goal: Driver creates a new account to access platform features.

Steps:

1. The driver opens the application and selects the “Sign Up” option.

2. The driver inputs their name, email, and vehicle license plate number.

3. The driver sets up a secure account password.

4. The system validates the entry data and creates a secure driver profile row in the database.

Use Case 2: Log in System

Actor: Driver

Goal: Driver logs into the application using active account credentials.

Steps:

1. The driver enters their registered email and password on the main landing screen.

2. The system checks the entries against secure password database tables.

3. The system authenticates the user and opens up the primary map menu environment.

Use Case 3: Search for Parking System

Actor: Driver

Goal: The driver identifies a specific parking location or lot of interest.

Steps:

1. The driver types A target street address or neighborhood name into the lookup search bar.

2. The system filters geographic facility arrays for lots within a short radius.

3. The screen populates interactive parking options on the display grid.

Use Case 4: Get viewing of real time parking display

Actor: Driver

Goal: Driver views the exact number of open parking spots available in real time

Steps:

1. The driver taps on an active parking facility listing icon

2. The system requests current live count inventory data from the target facilities' gateway sensors.

3. The application interface updates the display count matrix within one second.

Use Case 5: Interactive Viewing of Parking

Actor: Driver

Goal: Driver views a parking facility's precise location on a visual map grid interface

Steps:

1. The driver toggles the application map overlay settings switch.

2. The system connects to a third-party mapping engine API to render spatial data layout tiles

3. The user screen displays color-coded pins showing exact facility entry lines

Use Case 6: Search Parking Position with Sequential Filter

Actor: Driver

Goal: Driver analyzes parking data layouts using step-by-step logical price and distance filters.

Steps:

1. The driver opens up the advanced map lookup filtering panel interface tool

2. The driver then inputs a maximum acceptable hourly rate and a preferred maximum walking distance

3. The system processes the sequential filters and hides all lot options that do not meet the requirements.

Use Case 7: Reserve Parking Space

Actor: Driver

Goal: Driver works on a specific parking request to lock in a spot ahead of arrival

Steps:

1. The driver clicks the “book reservation” active key layout button on a target lot profile page

2. The driver enters their expected vehicle arrival hour and departure time frames

3. The system marks a spot inventory unit column as locked in the scheduling database table

Use Case 8: Reserve Reservation Confirmation

Actor: Driver

Goal: Driver waits for a parking notification update and receives a secure confirmation

Steps:

1. The platform engine verifies that the target parking space lock is active on the server arrays

2. The system generates a unique alphanumeric booking confirmation reference code number string

3. The mobile client sends a push notification banner confirmation message across the driver's screen window

Use Case 9: Direction to Parking System

Actor: Driver

Goal: Driver uses the application navigation system layers to drive directly to the reserved garage

Steps:

1. The driver taps the “Start Route Guidance” navigation launch control selection bar

2. The system extracts the geometric location markers from the active reservation ticket

3. The system launches an external mapping application framework to provide real-time turn-by-turn tracking lines

Use Case 10: Pay System

Actor: Driver

Goal: Driver initiates a digital payment process to complete their parking space purchase

Steps:

1. The driver clicks the final checkout action verification element to authorize account billing charges

2. The system routes a secure token package request out to a third-party credit card gateway provider agency

3. The processing network validates account funds and charges the parking fee balance successfully

Use Case 11: Access Reservation History System

Actor: Driver

Goal: Driver views a historical list of all previously paid parking spaces on their account profile

Steps:

1. The driver opens up their personal user dashboard and clicks on the transaction logs category index line

2. The system runs a search query against core system history tables matching the user reference ID

3. The interface window renders A scrolling chronological list detailing past garage reservations

Use Case 12: View Payment Receipt

Actor: Driver

Goal: driver views their formal transportation transaction tracking number payment total and date records

Steps:

1. The driver selects a specific finished trip row entry from the personal historical booking table index list

2. The system loads the associated digital invoice template schema from file backup nodes

3. The driver's screen populates a clean, printable breakdown displaying receipt details

Use Case 13: Notification Parking

Actor: Driver

Goal: Driver gets reminded by push alerts that their active reservation time window is closing soon

Steps:

1. The system server background workers keep a constant watch on expiration timelines inside the table indexes

2. The system identifies an active parking session with exactly 15 minutes left on the clock

3. The platform shoots a warning push reminder alert block straight to the driver's phone interface frame

Use Case 14: Log into Admin Dashboard System

Actor: Driver

Goal: Parking operators log into a custom management portal interface to view facility data fields

Steps:

1. The parking lot manager opens the corporate web app portal interface site and keys in staff credentials

2. The system checks data access profile configurations to make sure the user has executive access clearance

3. The server dashboard reveals administrative analytics windows, device settings, controls and pricing tools

Use Case 15: View Parking Occupancy Reports

Actor: Parking Operator

Goal: Parking operators view live parking status metrics and aggregate facility utilization chart data

Steps:

1. The facility operator clicks on the analytics and reporting option widget link from the manager sidebar menu

2. The system queries metrics covering real-time gate entry counts, exit loops, and peak vehicle cluster tracking lines

3. The web portal compiles data models into interactive summary charts for facility operational review.

IV. Agile Story Point Velocity Metrics

To estimate task complexity risk and relative effort an agile scrum estimation framework using the Fibonacci sequence in (1, 2,3,5,8) was applied to our Level 3 WBS items.

Our active Sprint 1 commitment focuses entirely on establishing foundational infrastructure and secure user entry points:

- UI-01 (Splash screen layout) -> 2 Story Points
- UI-02 (Sign-up form interfaces) -> 3 Story Points

- UI-03 (Login interface with error banners) -> 3 Story Points
- BE-01 (Database structural schemas and profile tables) -> 5 Story Points
- BE-02 (Input structural validation logic text) -> 3 Story Points
- BE-03 (Cryptographic password hashing utility) -> 3 Story Points
- BE-04 (JWT session validation token configurations) -> 3 Story Points

Total active Sprint 1 metric load: 24-story points

Projected solar velocity resource: 12-15 story points per week

Total accumulated backlog size: 145 story points (Estimated across 40 tasks).

Timeline alignment: 10-12 weeks' estimated duration, validating our December final production deployment target date.

Section 4.2 – Traditional COCOMO Mathematical Modeling

To validate our Agile metrics against traditional standards, the basic COCOMO (constructive cost model) in Organic Mode was applied, utilizing project constraints and modular dependencies.

1. **Baseline Parameters:**

- Estimated lines of code: 5000 lines
- KLOC: 5.0
- Mode Selection: Organic (Small team, familiar development environment)
- Constants Applied: ab = 2.4, bb = 1.05, cb = 2.5, db = 0.38

2. **Engineering Effort Calculations:**

i. Effort = ab x (KLOC) ^ bb
ii. Effort = 2.4 x (5.0) ^ 1.05
iii. Effort = 2.4 x 5.42 = 13.01 Person-Months

3. **Development Schedule Calculations:**

i. Time = cb x (Effort) ^ db
ii. Time = 2.5 x (13.01) ^ 0.38
iii. Time = 2.5 x 2.65 = 6.62 Months

4. **Academic Allocation Tweak Justification:**

While the pure math of a standard COCOMO environment yields an extended schedule length of 6.62 months, our target delivery timeline is accelerated to 3 calendar months. This acceleration is

fully achieved by operating as a highly single-focused resource and aggressively leveraging our third-party dependencies specifically pre-validated APIs for payment processing loop engines and navigation mapping layers.

V. Project Risk Management & Status Reporting

[Risk ID: R-01]

- Category: Technical / Integration Dependency
- Description: Third-party mapping and navigation API updates or billing
- Likelihood: Medium (3/5) , Impact: High (4/5), Critical Score: 12
- Mitigation Strategy: Implement a modular abstraction layer or wrapper class around the geographic map API. If the mapping endpoints change, modifications will be isolated to a single service file without breaking front-end UI-04.

[Risk ID: R-02]

- Category: Architectural / Database Security
- Description: Race conditions or transactional deadlocks occur when two drivers attempt to reserve the exact same parking spot concurrently (BE-09).
- Likelihood: Low (2/5), Impact: Critical (5/5), Critical Score: 10
- Mitigation Strategy: deploy strict row-level isolation and database transactions utilizing pessimistic row locking. If a spot is at mid-checkout, concurrent requests will safely receive an immediate “Spot Pending” transaction status message.

[Risk ID: R-03]

- Category: Project Schedule / Resource Constraint
- Description: Operating as a single-resource solo developer causes an engineering bottleneck if a single complex backend milestone (like Stripe API gateway setup) takes longer than its 1-week allocated sprint cycle.
- Likelihood: High (4/5), Impact: Medium (3/5), Critical Score: 12
- Mitigation Strategy: Maintain a strict 20% point buffer within our backlog planning. If velocity metrics drop below 15 points per week, secondary reporting features (REP-09, REP-10) will be deprioritized to preserve core reservation mechanics.

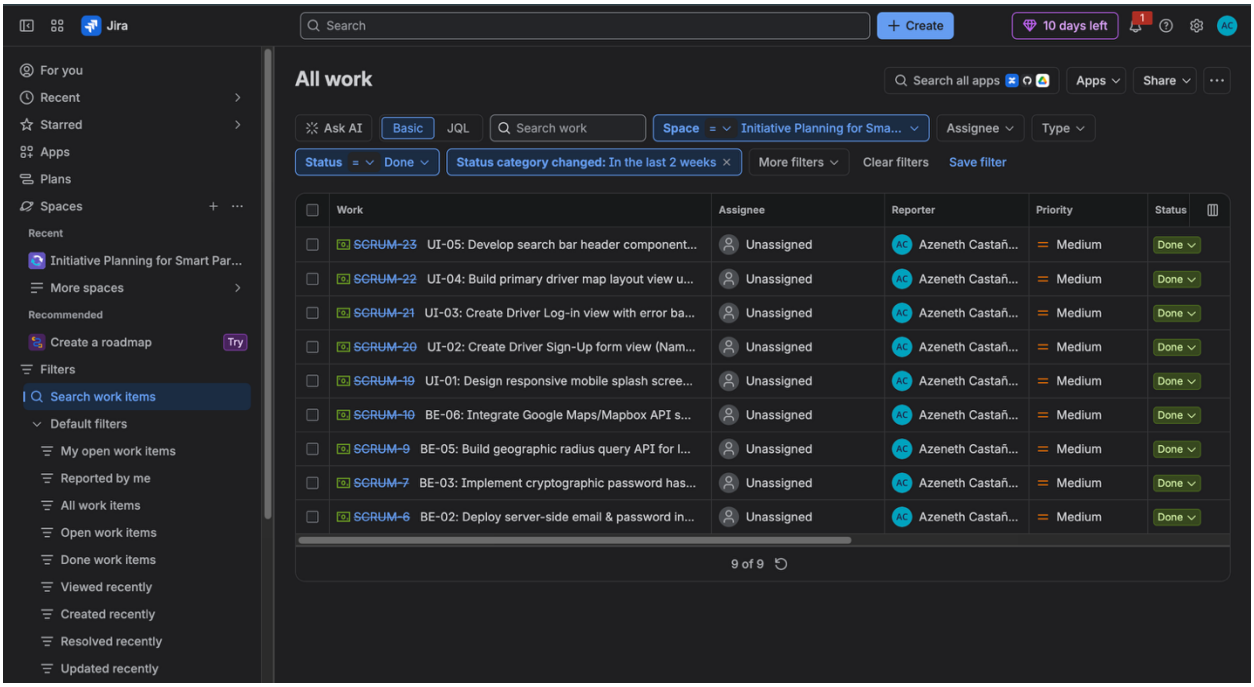
VI. Mid-Semester Engineering Audit & Sprint Review

As of October 15, 2026, a comprehensive mid-semester audit was completed to verify the structural synchronization between the master project schedule, functional use cases, and the live Agile Scrum environment in Jira.

Sprint 2 Evaluation & Outcomes:

The second development cycle has formally concluded with a recorded velocity of 21 completed story points. The following core user interface and architectural modules have been completed, verified, and closed out:

- UI-04: Driver Interactive Map layout View Component (5 Story Points)
- UI-05: Street Address Autocomplete Search Bar Header (3 Story Points)
- BE-05: Geographic Radius Query DB API Endpoint Logic (5 Story Points)
- BE-06: Google Maps/Mapbox API Spatial Tile Hookups (8 Story Points)



Architectural Review & Future Outlook:

To protect front-end rendering performance, a critical design choice was made to isolate our third-party mapping webhooks within a dedicated back-end abstraction layer, mitigating risk parameters regarding API threshold adjustments.

A total of 9 core engineering tasks have now been successfully marked as done across our project history; the active environment has been synchronized, and the remaining uncompleted profile authentication tasks (SCRUM-5 and SCRUM-8) have automatically rolled forward into the upcoming Sprint 3 container to maintain the sequential momentum.

VII. Conclusion

The first version of the project document gives an overview of the Smart Parking Platform, makes a difference between the Smart Parking Platform, the parking solutions already present, starts to

elaborate The Vision and Scope section and offers a draft of the Requirement Specification Document which features 15 use cases. Later revisions will include more requirements, a more detailed view of stakeholders, and thorough project planning.